

MMD GANs

Mahdi Bouafif Rayene Tahar Nour Ghamgui
Mohamed Boudouh

Télécom Paris — Institut Polytechnique de Paris

Encadrant: Arthur Leclaire

Juin 2026



Outline

- 1 GANs, WGANs, MMD Gans
- 2 Expériences sur MNIST
- 3 Résultats sur CelebA
- 4 Exploration (Exploitation) de l'espace latent du bruit

Generative Adversarial Networks

GANs

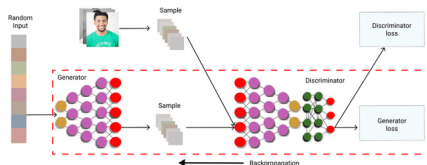
A GAN is a couple of two networks G_ψ and D_θ :

The **generator** : $G_\psi : \mathcal{Z} \rightarrow \mathcal{X}$ is trained to map a fixed input distribution (generally a white noise $z \sim \mathcal{N}(0, I)$) to a target data distribution Q .

The **discriminator** : D_θ is trained to separate the generated distribution during the training from the target distribution.

A GAN is trained by solving a **min-max problem**: $\min_\psi \max_\theta D_\theta(P, Q_\psi)$;

$G_\psi(z) \sim Q_\psi$



Different ways to compare Probability distributions

GAN Variants

Metric	Model
JS Div.	(Vanilla) GAN
Optimal transport	WGAN
MMD	MMD-GAN

Maximum Mean Descreapancy

$$\text{MMD}^2(P, Q; \mathcal{H}_k) = \mathbb{E}_{X, X'}[k(X, X')] + \mathbb{E}_{Y, Y'}[k(Y, Y')] - 2\mathbb{E}_{X, Y}[k(X, Y)]$$

$k(x, y)$ is generally chosen as a simple decreasing function of $\|x - y\|^2$:

RBF: $k(x, y) = e^{\frac{-\|x-y\|^2}{2\sigma^2}}$; **RQ**: $k(x, y) = (1 + \frac{\|x-y\|^2}{2\alpha l})^{-\alpha}$

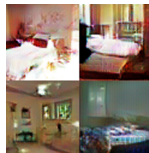
In general k has to be a **positive definite kernel** and $X, X' \sim P, Y, Y' \sim Q$.

Why MMD Gans?

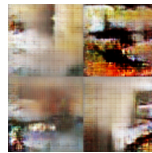
- **Smaller feature extraction network** sufficient for MMD-GAN to reach better visual results than WGANs.

- **Unbiased gradient estimator:**

$$\mathbb{E}[\nabla_{\psi} \widehat{\text{MMD}}_u^2] = \nabla_{\psi} \text{MMD}^2.$$



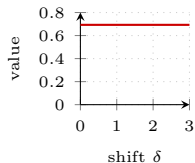
MMD GAN
dimension of
feature space
16



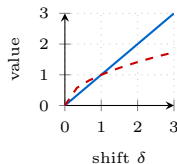
WGAN GP
dimension of
feature space
16

Distance between 1D
distributions

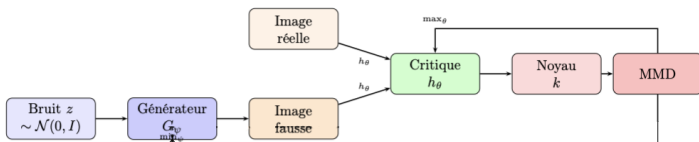
JS Divergence



Wasserstein-1 / MMD



Architecture MMD-GAN



Pipeline

- **Générateur** G_ψ : transforme un bruit $z \sim \mathcal{N}(0, I)$ en image fausse , minimise la MMD
- **Critique** h_θ : projette images réelles et fausses dans un espace de features , maximise la MMD
- **Noyau** k : calcule la MMD entre les features réelles et fausses

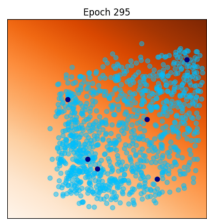
$$\text{MMD}(\mathbb{P}, \mathbb{Q}; \mathcal{H}) = \sup_{f \in \mathcal{H}, \|f\|_{\mathcal{H}} \leq 1} \mathbb{E}_{\mathbb{P}} f(X) - \mathbb{E}_{\mathbb{Q}} f(Y)$$

$$f^*(x) \propto \mathbb{E}_{\mathbb{P}} k(X, x) - \mathbb{E}_{\mathbb{Q}} k(Y, x)$$

Bande passante du noyau — Données synthétiques 2D

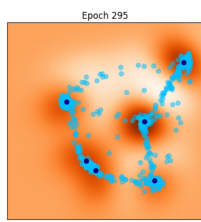
$$k(x, y) = e^{\frac{-\|x-y\|^2}{2\sigma^2}}$$

$$f^*(x) \propto \mathbb{E}_{\mathbb{P}} k(X, x) - \mathbb{E}_{\mathbb{Q}} k(Y, x)$$

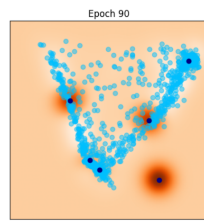


$\sigma = 2$

$\nabla_x k$ non nul partout
mais $\|\nabla_x k\| \ll 1$.



$\sigma = 0.1$



$\sigma = 0.05$

Fort $\nabla_x k$ autour des
données d'origine mais
très-local.

Pouvoir discriminant vs. Étendue spatiale

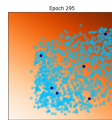
Proposition (Compromis bande passante–discriminabilité)

Pour des noyaux de la forme
 $k(x, y) = f(\|x - y\|^2)$, $f(0) = 1$
on définit la **bande passante- d**
comme

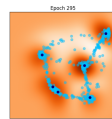
$\sigma_d = \sup\{x \mid f'(x) < -d\}$. Alors
:

$$\sigma_d \cdot d \leq 1$$

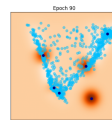
*Il est impossible d'avoir
simultanément un gradient à la
fois fort et spatialement étendu.*



$\sigma = 2$: diffus, $\|\nabla k\|$ faible



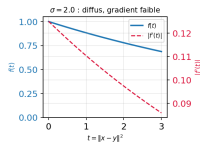
$\sigma = 0.1$: bon compromis



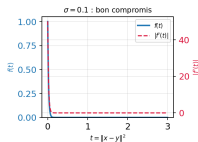
$\sigma = 0.05$: fort mais hyper-local

Visualiser la dérivée en 1D

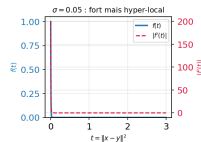
$$f(t) = e^{\frac{-t^2}{2\sigma^2}}$$



$\sigma = 2$



$\sigma = 0.1$



$\sigma = 0.05$

Suivi des hyperparamètres — ACP 3D, moyenner des Noyaux

Visualisation PCA 3D

Nous projetons les features sur les **3 composantes principales** de la matrice de covariance associée à la matrice de données jointe $X = \begin{bmatrix} \text{features réelles} \\ \text{features générées} \end{bmatrix}$

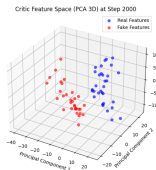
Choix de la bande passante

Nous pouvons utiliser un noyau *mélange* qui combine plusieurs échelles :

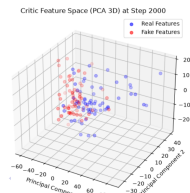
$$k(x, y) = \sum_{k=1}^K k_{\sigma_k}(x, y).$$

La MMD reste sensible tant qu'un des σ_k correspond à l'échelle actuelle des caractéristiques

Critique en avance sur le générateur



Les features générées se rapprochent de celles réelles



Expériences sur MNIST

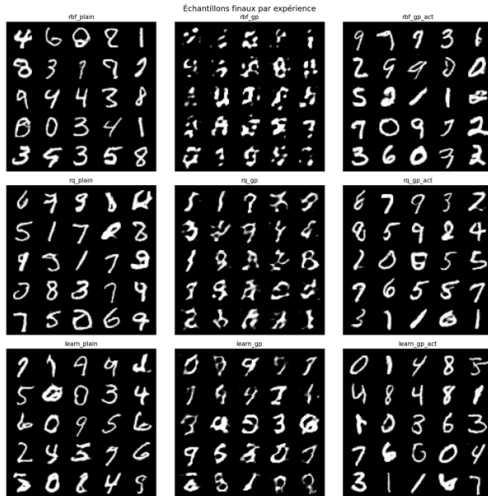


Figure: Résultats d'expériences sur MNIST

Commentaires sur les expériences sur MNIST

- **Noyau RBF** ($\sigma = 1$) : Chiffres très nets et reconnaissables, MMD^2 descend de 0.98 à 0.21 : un signal exploitable toujours. Même sans contrainte le modèle est **stable**. Pas de saturation du RBF à cette échelle.
- **RBF avec Gradient Penalty** : Dégradation forte, l'act grimpe jusqu'à 1455 alors qu'il n'y a pas de pénalité pour le borner. Le GP (final : 0.04) contraint la pente de la fonction témoin en forçant des features à faible échelle, ce qui sature le noyau RBF et casse le gradient.
- **rbf_gp_act** Le meilleur résultat entre les 3. Les logs sont stables

Commentaires sur les expériences sur MNIST

- **Noyau RQ**($\sigma = 1$) : Chiffres nets et reconnaissables, Mais MMD^2 diverge complètement. Cette expérience a probablement bien fonctionné jusqu'à un certain point puis a explosé après 25 000
- **RQ avec Gradient Penalty** : a GP seule ne suffit pas à borner les features avec le noyau RQ, act grimpe à 130 206 en fin d'entraînement
- **RQ_gp_act** Chiffres nets et bien formés, reste borné (max 1.05, final 0.109), MMD^2 finale 0.58 (convergence active, pas de saturation).

Commentaires sur les expériences sur MNIST

- **learn_plain** : Résultats bons à très bons, chiffres lisibles. Les logs montrent que les α s'effondrent tous vers 0.05 qui est un comportement d'autorégularisation implicite. le noyau réduit ses propres bandwidths pour compenser l'absence de contrainte externe.
- **learn_gp** : Qualité plus irrégulière, certains chiffres ambigus. Act grimpe à 49 058 en fin d'entraînement (non borné) ce qui explique le bruit visuel.
- **learn_gp_act** Excellent, parmi les meilleurs de toute la grille, act reste strictement borné (max 1.2, final 0.088), GP stable (0.18), et les α convergent.

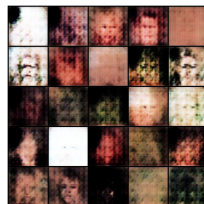
CelebA — Le noyau RBF sature

Constat

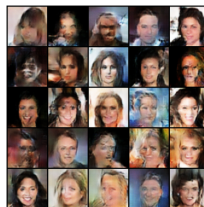
- **Sans régularisation** : la MMD s'écrase sur le plafond dès l'itération $\sim 6000 \rightarrow$ taches de couleur, pas de structure de visage.
- **GP seule** : ne règle rien, la saturation persiste et la GP finit par s'emballer.
- **GP + activation penalty** : stabilise l'entraînement, mais visages flous et peu diversifiés.

Pourquoi?

$\sigma = 1$ fixe $\Rightarrow$ portée trop étroite pour la dimensionnalité de CelebA (64×64 couleur) $\Rightarrow$ pénalise la diversité visuelle.



rbf_plain – aucune structure



rbf_gp_act – stable mais flou

CelebA — RQ^* fixe vs. RQ^* appris

RQ^* fixe (exp. 4–6)

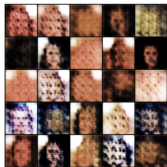
Même classement que sur MNIST :

- exp. 4 (sans GP) : instable
- exp. 5 (GP seule) : partielle
- **exp. 6 (GP + act)** : la plus stable et diversifiée

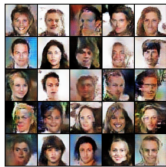
Décroissance polynomiale $\Rightarrow$ pas de saturation même à grande échelle de features.

RQ^* appris (exp. 7–9)

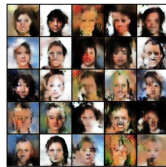
- Sans régularisation : fortement instable.
- GP seule : améliore mais **dégrade en fin d'entraînement.**
- **GP + act** : comportement le plus robuste, limite la dérive des α .



RQ^* instable



RQ^* GP+act



appris GP+act

CelebA — Ce qu'on en retient

Effet de la dimensionnalité

L'explosion du gradient penalty sans régularisation est **bien plus sévère sur CelebA que sur MNIST** : confirme que l'instabilité s'aggrave avec la dimension des données.

Évolution des bandwidths α

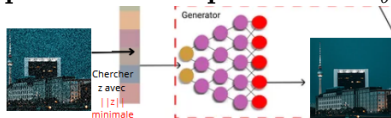
- Sans GP : les α se contractent.
- GP seule : dérive vers le haut, en phase avec les sursauts de GP.
- **Activation penalty : contient cette dérive.**

Conclusion

GP + activation penalty reste la configuration la plus robuste sur CelebA comme sur MNIST. Le noyau RQ* (fixe ou appris) domine largement le RBF dès que la dimension augmente.

Restauration profonde en utilisant un MMD-GAN préentraîné

Choix de $z \in Z$
(convenable) avec $\|z\|$ faible
puis Forward pass dans G_θ



Restauration par méthodes variationnelles

On propose de résoudre un problème de restauration en minimisant une énergie du type :

$$\hat{x} \in \arg \min_x E(x) = f(y, x) + \lambda R(x)$$

Exploitation du prior appris par le générateur G_θ

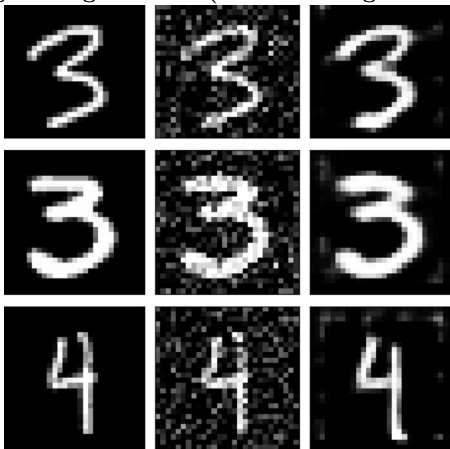
Supposons que l'image qu'on souhaite restaurer appartient à une distribution de probabilité apprise par le générateur G_θ d'un MMD GAN . On peut exploiter ce prior en l'encodant dans R

Implémentation

On cherche une image proche de y mais qui reste une sortie probable de G_θ
La formulation de la restauration devient
$$\hat{z} \in \arg \min_z f(y, G_\theta(z)) + \frac{\lambda}{2} \|z\|^2,$$
$$\hat{x} = G_\theta(\hat{z}).$$

Débruitage sur Mnist (Bruit additif gaussien)

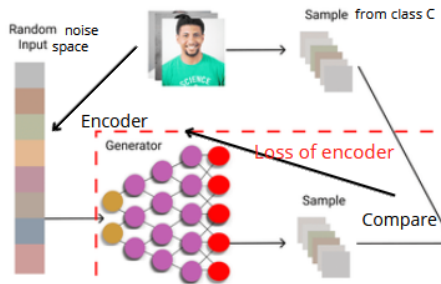
Débruitage de digits Mnist (Déscente de gradient sur $E(y)$)



Conditionner le générateur en encodant des classes dans l'espace latent Z

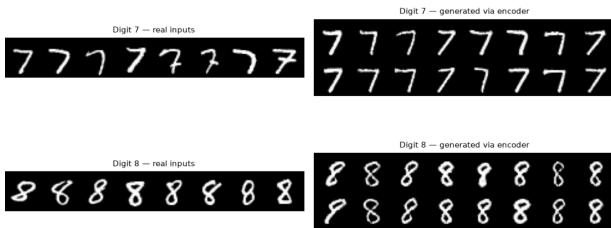
On suppose qu'on dispose d'un générateur de MMD GAN préentraîné

Encoder une classe dans Z

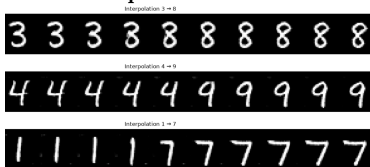


Conditionner le générateur en encodant des classes dans l'espace latent Z

Génération conditionnée sur Mnist après encodage dans Z



Interpolation dans Z



References I



I. Goodfellow et al. *Generative Adversarial Nets*. NeurIPS 2014. arXiv:1406.2661.



M. Bińkowski, D. J. Sutherland, M. Arbel, A. Gretton. *Demystifying MMD GANs*. ICLR 2018. arXiv:1706.08500.



M. Arjovsky, S. Chintala, L. Bottou. *Wasserstein Generative Adversarial Networks*. ICML 2017.



I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, A. Courville. *Improved Training of Wasserstein GANs*. NeurIPS 2017.



A. Gretton, K. Borgwardt, M. Rasch, B. Schölkopf, A. Smola. *A Kernel Two-Sample Test*. JMLR 13, 2012.



A. Müller. *Integral Probability Metrics and Their Generating Classes of Functions*. Advances in Applied Probability 29(2), 1997.